
Documentação de Projeto

para o sistema

GymFlow

Versão 1.0

Sistema de Treinos Personalizados e Acompanhamento Físico

Projeto de sistema elaborado pelo aluno(Jonas Martins
como parte da disciplina **Projeto de Software**.

2026

Tabela de Conteúdo

1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelo de Casos de Uso e Histórias de Usuários	1
2.3 Diagrama de Sequência do Sistema e Contrato de Operações	1
3. Modelos de Projeto	1
3.1 Arquitetura	1
3.2 Diagrama de Componentes e Implantação.	2
3.3 Diagrama de Classes	2
3.4 Diagramas de Sequência	2
3.5 Diagramas de Comunicação	2
3.6 Diagramas de Estados	2
4. Modelos de Dados	2

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Jonas Martis	10/06/2026	Revisão de Controle do Trabalho Apresentado	1.0

1. Introdução

Este documento agrega: 1) a elaboração e revisão de modelos de domínio e 2) modelos de projeto para o sistema GymFlow. A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema.

O GymFlow é um sistema voltado ao gerenciamento de treinos personalizados e acompanhamento físico em academias. Permite que alunos registrem suas sessões de treino, acompanhem sua evolução física ao longo do tempo e visualizem suas fichas de exercícios. Recepcionistas realizam o cadastro de alunos e gerenciam planos de treino, enquanto administradores têm visibilidade completa sobre o sistema.

O sistema adota arquitetura de microsserviços com Spring Boot, banco de dados PostgreSQL e frontend React, totalizando 5 microsserviços independentes comunicando-se via API Gateway (Spring Cloud Gateway).

2. Modelos de Usuário e Requisitos

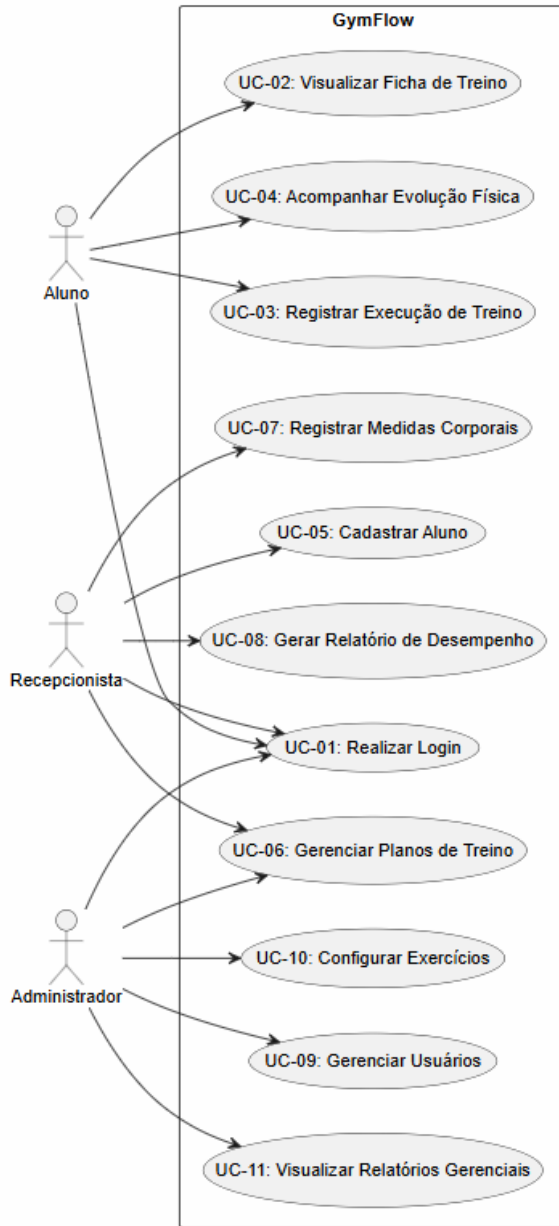
2.1 Descrição de Atores

Ator	Descrição
Aluno	Usuário matriculado na academia. Acessa o sistema para visualizar fichas de treino, registrar execuções, acompanhar evolução física e histórico de medidas corporais.
Recepcionista	Funcionário responsável pelo atendimento. Realiza cadastro de alunos, gerencia planos de treino, registra medidas corporais e gera relatórios de desempenho.
Administrador	Gestor do sistema com acesso total. Gerencia todos os usuários, configura o catálogo de exercícios, visualiza relatórios gerenciais e supervisiona planos de treino.

2.2 Modelo de Casos de Uso

ID	Caso de Uso	Ator Principal	Prioridade
----	-------------	----------------	------------

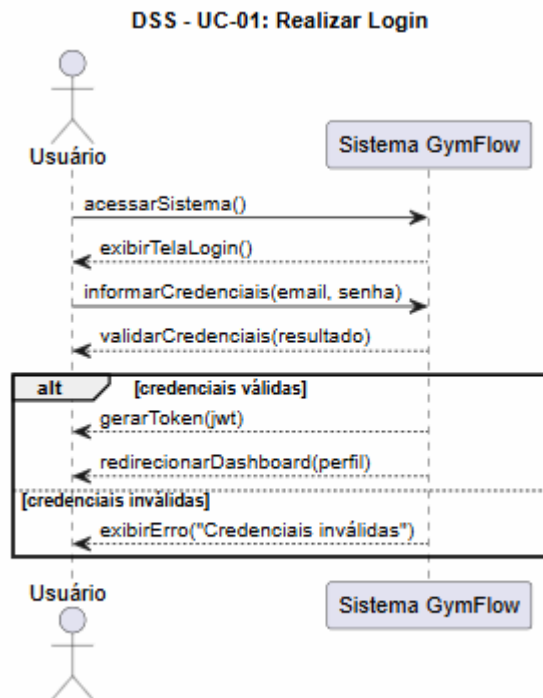
UC-01	Realizar Login	Todos	Alta
UC-02	Visualizar Ficha de Treino	Aluno	Alta
UC-03	Registrar Execução de Treino	Aluno	Alta
UC-04	Acompanhar Evolução Física	Aluno	Alta
UC-05	Cadastrar Aluno	Recepcionista	Alta
UC-06	Gerenciar Planos de Treino	Recepcionista / Administrador	Alta
UC-07	Registrar Medidas Corporais	Recepcionista	Média
UC-08	Gerar Relatório de Desempenho	Recepcionista	Média
UC-09	Gerenciar Usuários	Administrador	Alta
UC-10	Configurar Exercícios	Administrador	Média
UC-11	Visualizar Relatórios Gerenciais	Administrador	Média



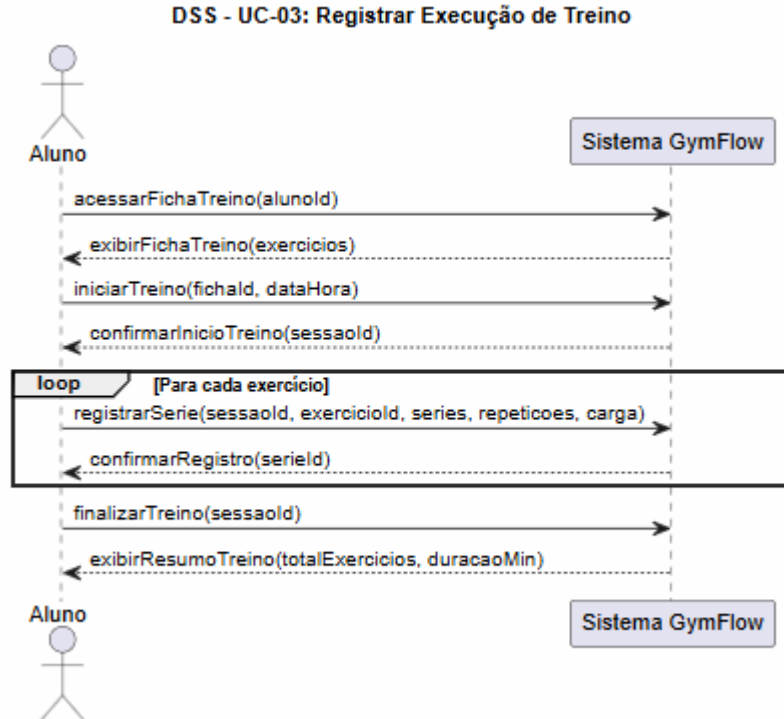
2.3 de Sequência do Sistema

Diagrama

Esta subseção apresenta os diagramas de sequência do sistema para os principais casos de uso, seguidos de seus contratos de operação.

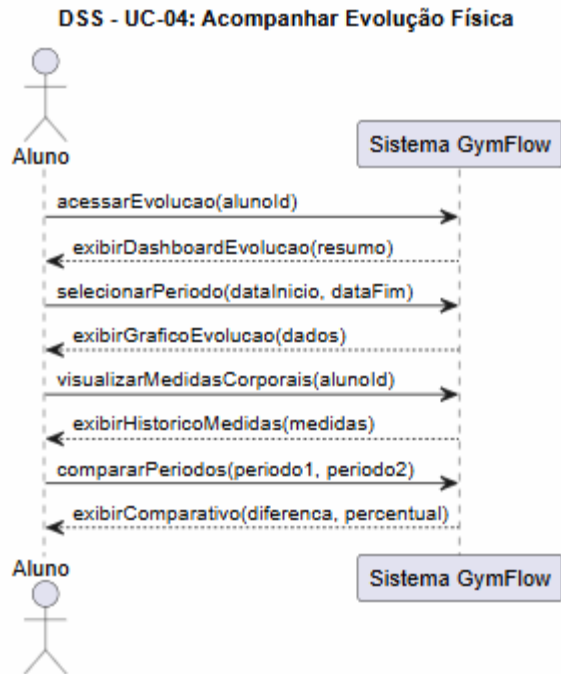
DSS — UC-01: Realizar Login**Contrato — informarCredenciais()**

Contrato	
Operação	informarCredenciais(email, senha)
Referências cruzadas	UC-01
Pré-condições	O sistema está acessível. O usuário não está autenticado.
Pós-condições	O sistema valida as credenciais contra o banco. Se válidas, um token JWT é gerado com o perfil do usuário e retornado ao cliente. O usuário é redirecionado ao dashboard correspondente ao seu perfil.

DSS — UC-03: Registrar Execução de Treino**Contrato — registrarSerie()**

Contrato	
Operação	registrarSerie(sessaoid, exercicioFichaid, series, repeticoes, carga)
Referências cruzadas	UC-03
Pré-condições	O aluno está autenticado. A sessão de treino está com status EM_ANDAMENTO. O exercicioFichaid pertence à ficha vinculada à sessão.
Pós-condições	Um novo objeto SerieRegistrada é criado e persistido associado à sessão. Os dados de séries, repetições e carga são salvos. O progresso da sessão é atualizado.

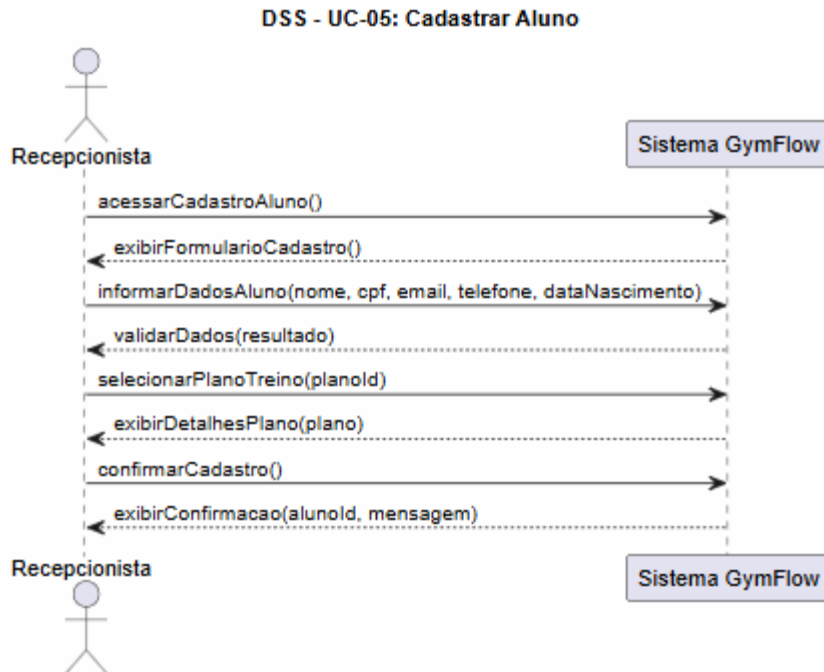
DSS — UC-04: Acompanhar Evolução Física



Contrato — selecionarPeriodo()

Contrato	
Operação	selecionarPeriodo(dataInicio, dataFim)
Referências cruzadas	UC-04
Pré-condições	O aluno está autenticado. Existem sessões registradas no período informado.
Pós-condições	O sistema filtra e agrega as sessões do período. Calcula métricas de volume, frequência e evolução de cargas. Retorna os dados formatados para exibição em gráfico.

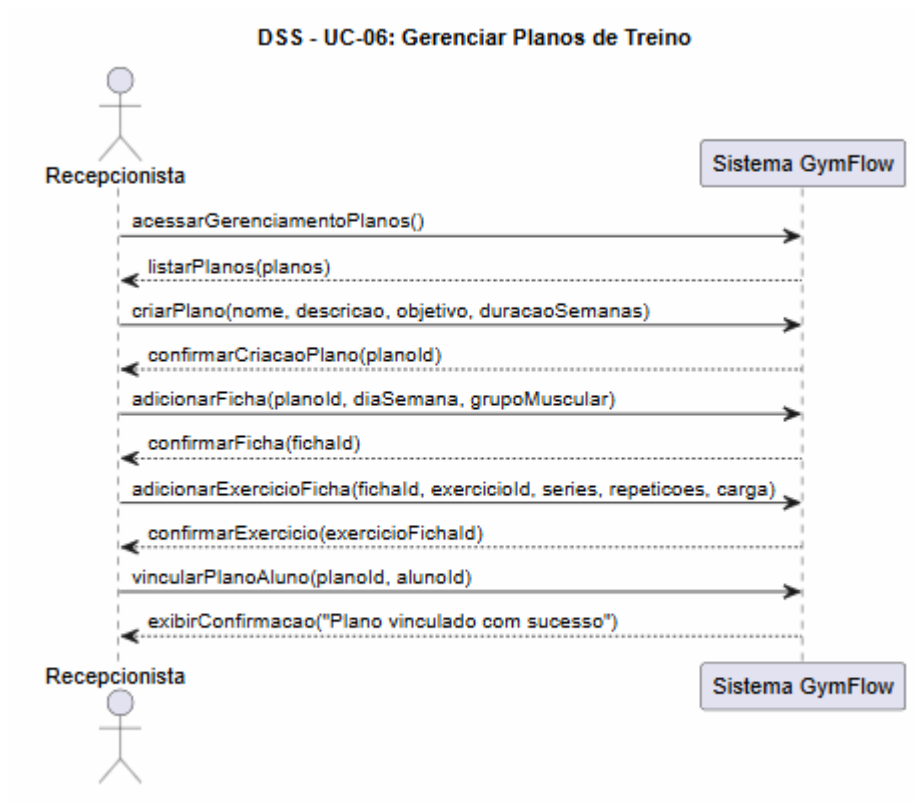
DSS — UC-05: Cadastrar Aluno



Contrato — informarDadosAluno()

Contrato	
Operação	informarDadosAluno(nome, cpf, email, telefone, dataNascimento)
Referências cruzadas	UC-05
Pré-condições	A recepcionista está autenticada. Não existe usuário com o CPF ou e-mail informado no sistema.
Pós-condições	Um novo Usuario é criado com tipo ALUNO. Um objeto Aluno é vinculado ao usuário. Status definido como ATIVO. Senha temporária gerada e enviada por e-mail.

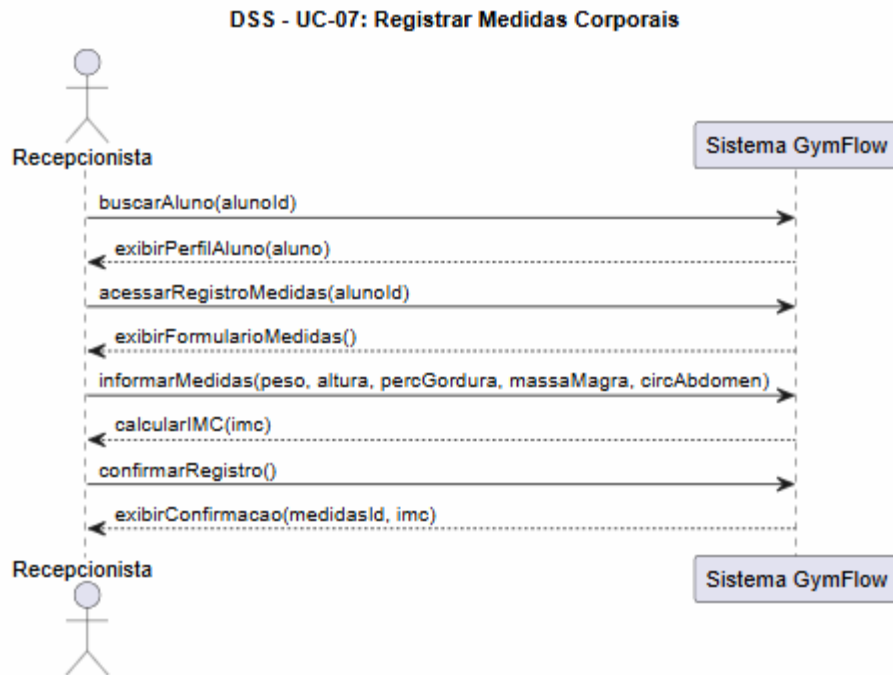
DSS — UC-06: Gerenciar Planos de Treino



Contrato — adicionarExercicioFicha()

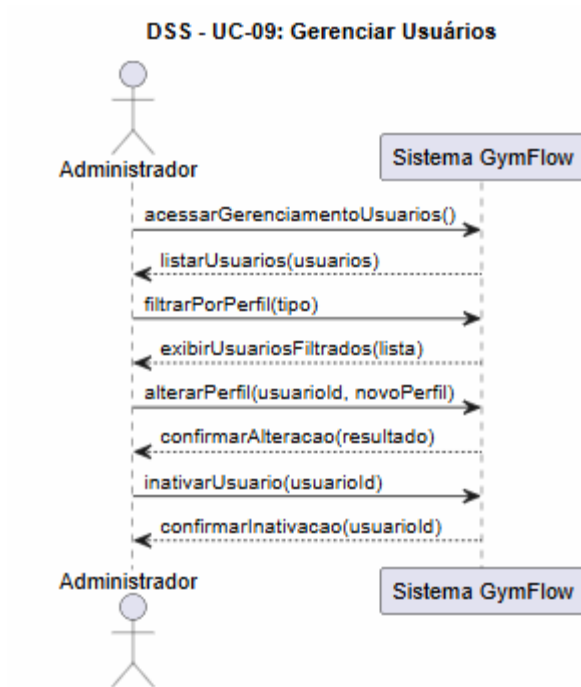
Contrato	
Operação	adicionarExercicioFicha(fichald, exerciciold, series, repeticoes, carga)
Referências cruzadas	UC-06
Pré-condições	A recepcionista está autenticada. A ficha de treino pertence a um plano existente. O exerciciold corresponde a um exercício cadastrado no catálogo.
Pós-condições	Um novo ExercicioFicha é criado associando o exercício à ficha com os parâmetros de séries, repetições e carga sugerida. A ordem do exercício na ficha é atribuída automaticamente.

DSS — UC-07: Registrar Medidas Corporais

**Contrato — informarMedidas()**

Contrato	
Operação	informarMedidas(peso, altura, percGordura, massaMagra, circAbdomen)
Referências cruzadas	UC-07
Pré-condições	A recepcionista está autenticada. O alunoid corresponde a um aluno ATIVO no sistema.
Pós-condições	Um novo objeto MedidaCorporal é criado com a data atual. O IMC é calculado automaticamente ($\text{peso}/\text{altura}^2$). Os dados são persistidos e vinculados ao histórico do aluno.

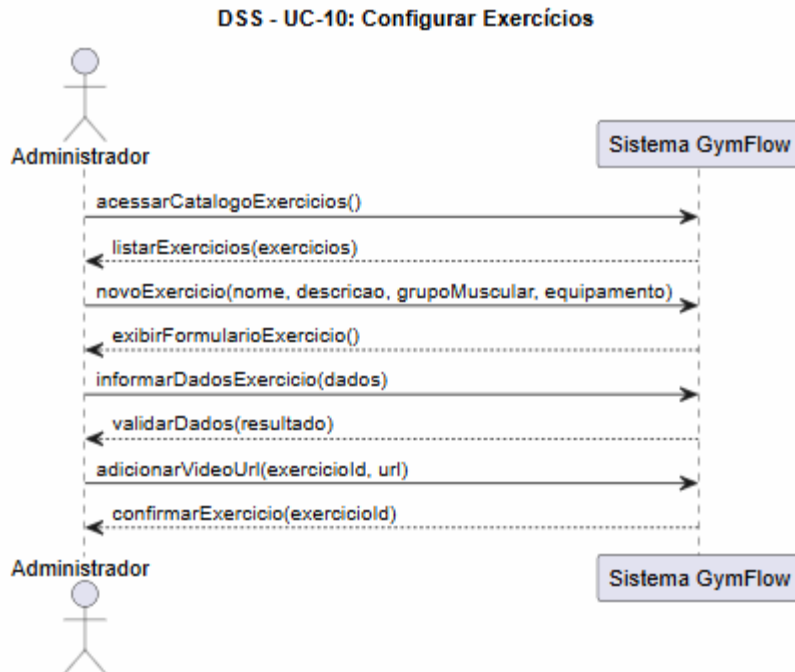
DSS — UC-09: Gerenciar Usuários



Contrato — inativarUsuario()

Contrato	
Operação	inativarUsuario(usuarioId)
Referências cruzadas	UC-09
Pré-condições	O administrador está autenticado. O usuário alvo existe e está ATIVO. O administrador não pode inativar a si mesmo.
Pós-condições	O status do usuário é alterado para INATIVO. O token JWT vigente do usuário é invalidado. O usuário não consegue mais realizar login até ser reativado.

DSS — UC-10: Configurar Exercícios



Contrato — informarDadosExercicio()

Contrato	
Operação	informarDadosExercicio(nome, descricao, grupoMuscular, equipamento, videoUrl)
Referências cruzadas	UC-10
Pré-condições	O administrador está autenticado. Não existe exercício com o mesmo nome no catálogo.
Pós-condições	Um novo objeto Exercicio é criado e persistido no catálogo. O exercício fica disponível para seleção nas fichas de treino.

3. Modelos de Projeto

3.1 Arquitetura

O GymFlow adota arquitetura de microsserviços, onde cada domínio de negócio é um serviço Spring Boot independente com seu próprio banco de dados PostgreSQL, seguindo o padrão

Database per Service. A comunicação entre serviços é feita via HTTP REST roteada por um API Gateway (Spring Cloud Gateway). O frontend React comunica-se exclusivamente com o gateway.

Microserviço	Porta	Responsabilidade
Auth Service	8081	Autenticação, geração e validação de tokens JWT
Aluno Service	8082	Cadastro, perfil e gerenciamento de alunos
Treino Service	8083	Fichas de treino, exercícios e sessões
Evolução Service	8084	Medidas corporais, histórico e relatórios de evolução
Notificação Service	8085	Envio de e-mails e notificações ao aluno

Padrões de projeto adotados na arquitetura:

- Repository Pattern: isolamento da camada de persistência via Spring Data JPA.
- DTO Pattern: transferência de dados entre camadas sem expor entidades JPA.
- Service Layer: regras de negócio encapsuladas em classes de serviço.
- API Gateway Pattern: ponto único de entrada para o frontend, com roteamento e autenticação centralizada.
- Database per Service: cada microserviço possui seu próprio schema PostgreSQL isolado.

3.2 Diagrama de Componentes e Implantação.

Diagrama de Componentes

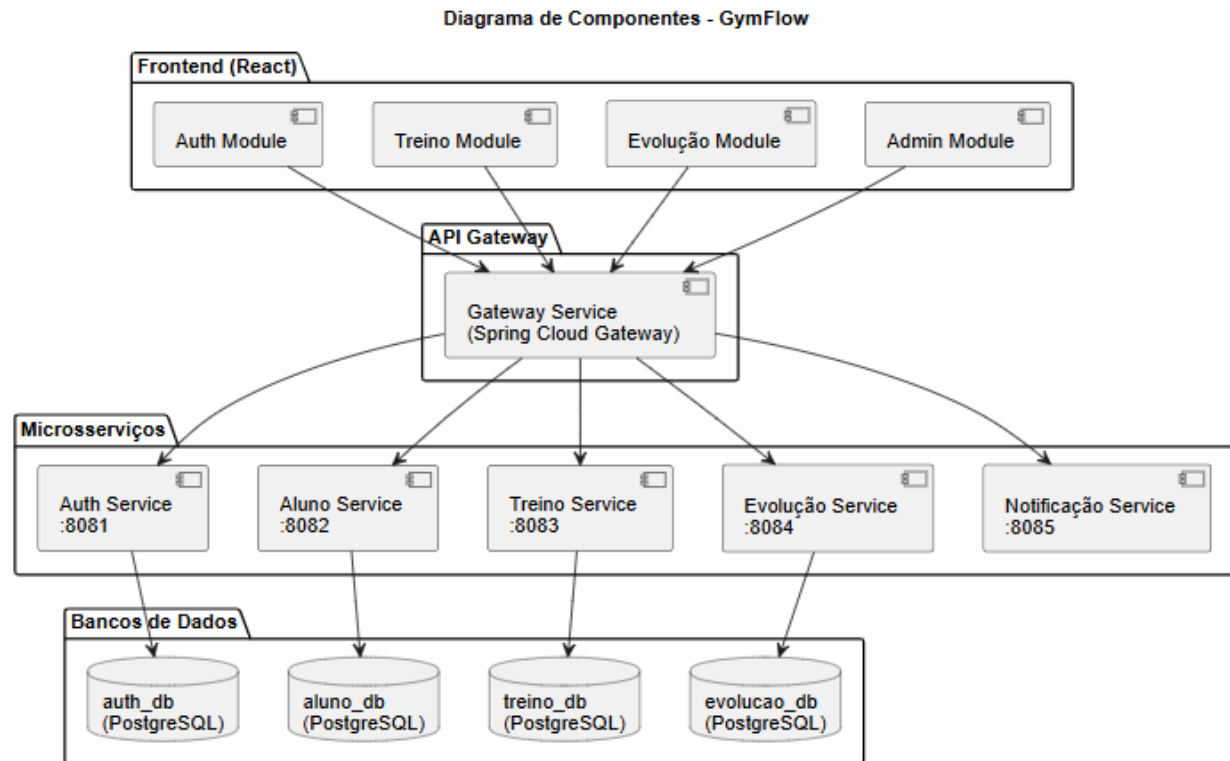
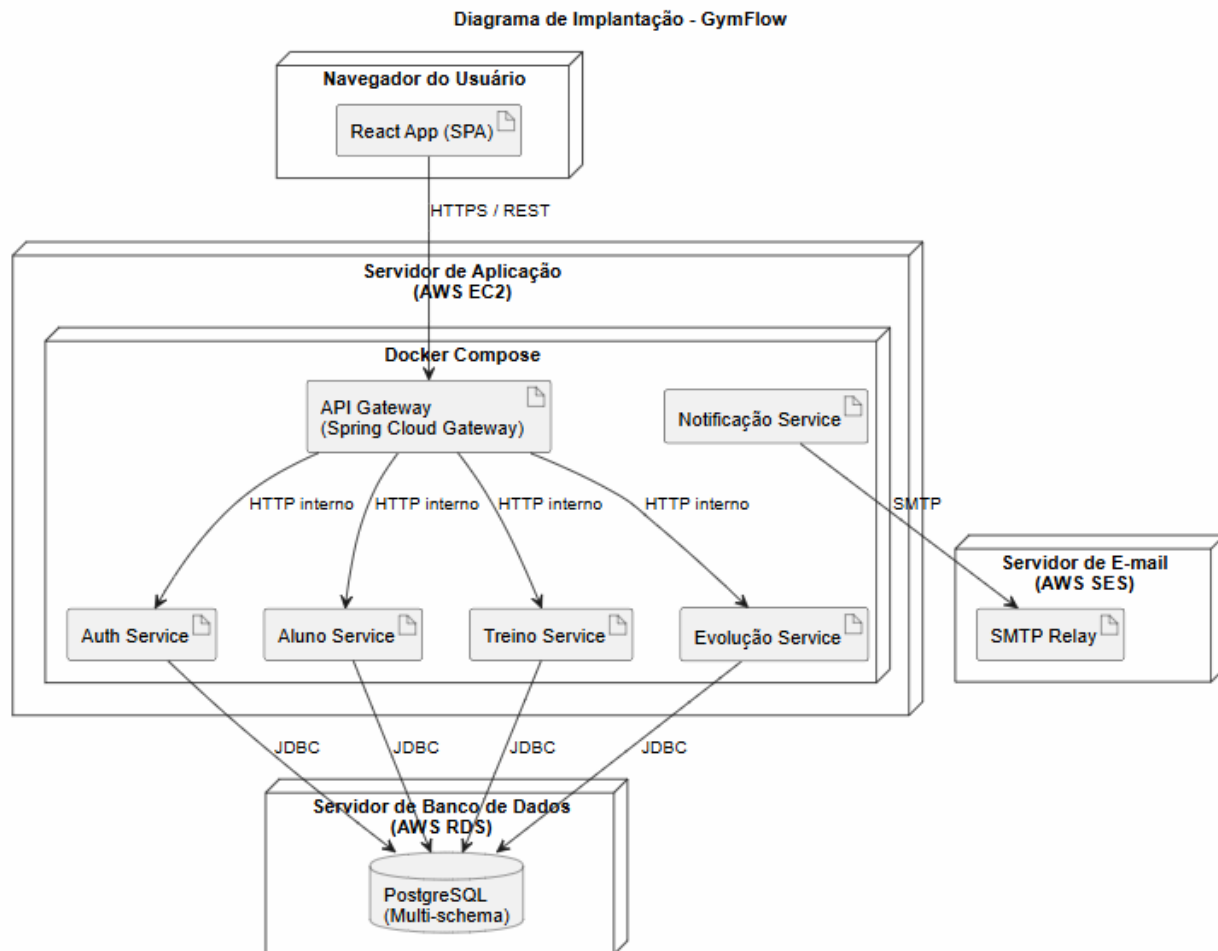


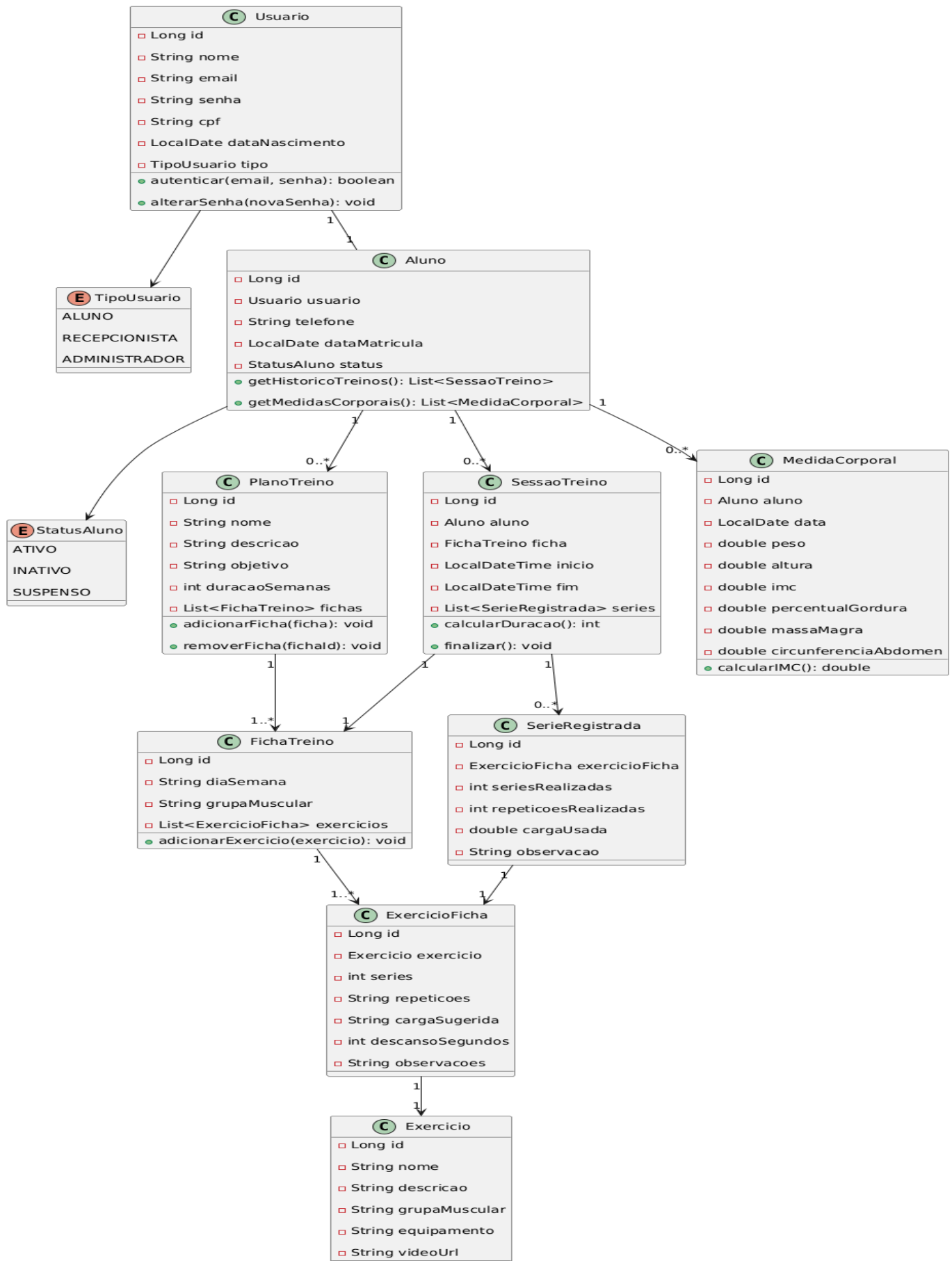
Diagrama de Implantação



3.3 Diagrama de Classes

O diagrama de classes abaixo representa as entidades principais do domínio GymFlow com seus atributos, operações e relacionamentos. **(PRÓXIMA PAGINA)**

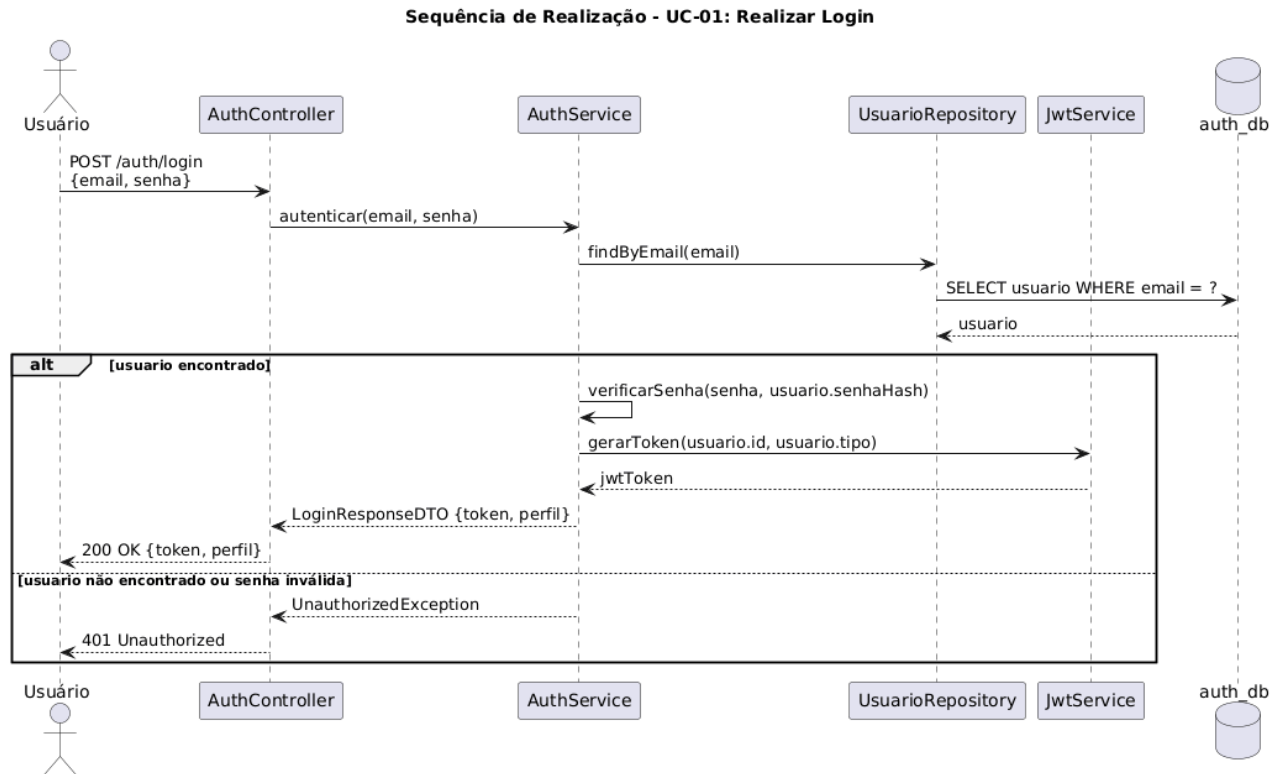
Diagrama de Classes - GymFlow



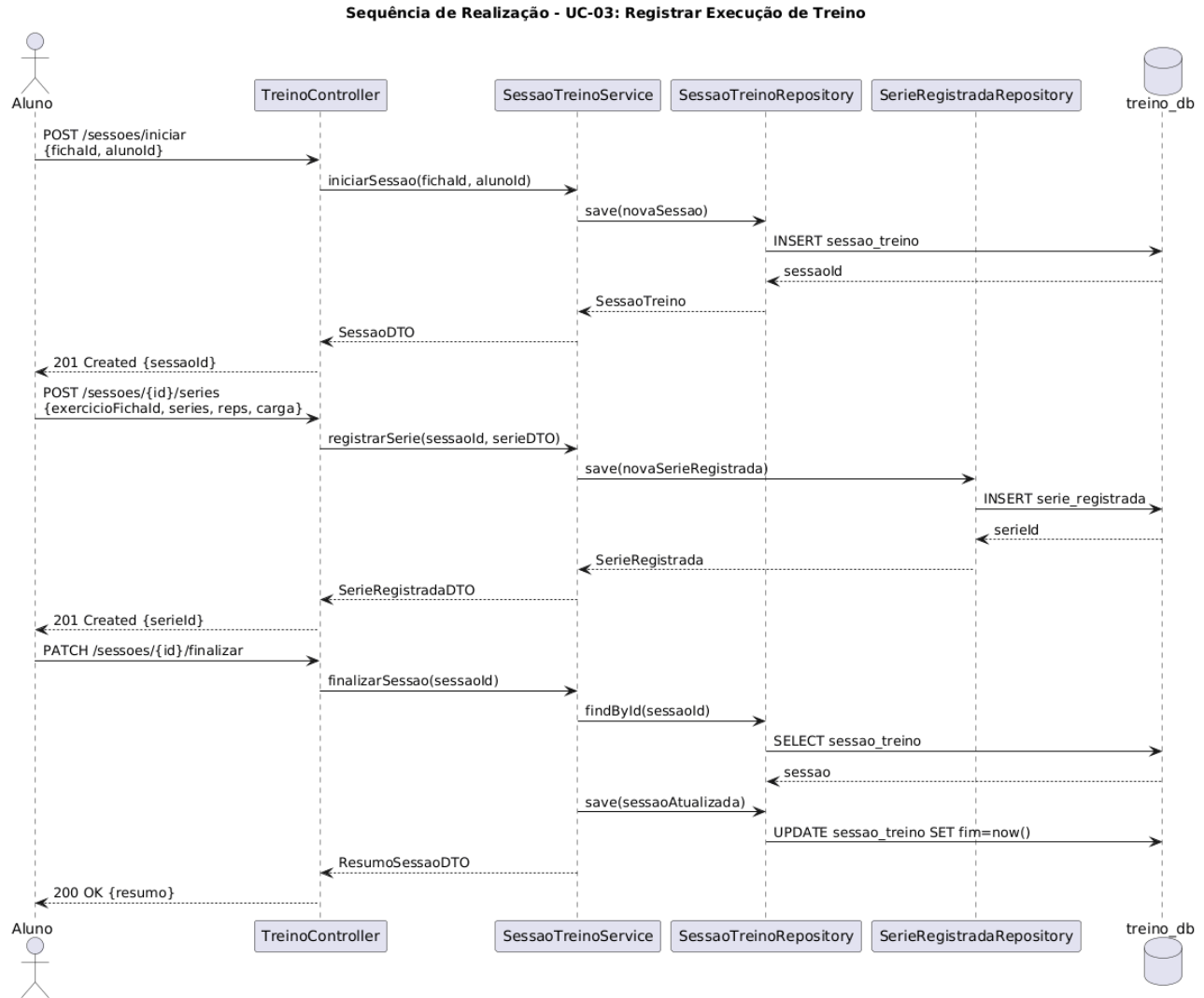
3.4 Diagramas de Sequência

Diagramas de sequência detalhados para realização dos casos de uso, mostrando a interação entre as camadas Controller, Service e Repository.

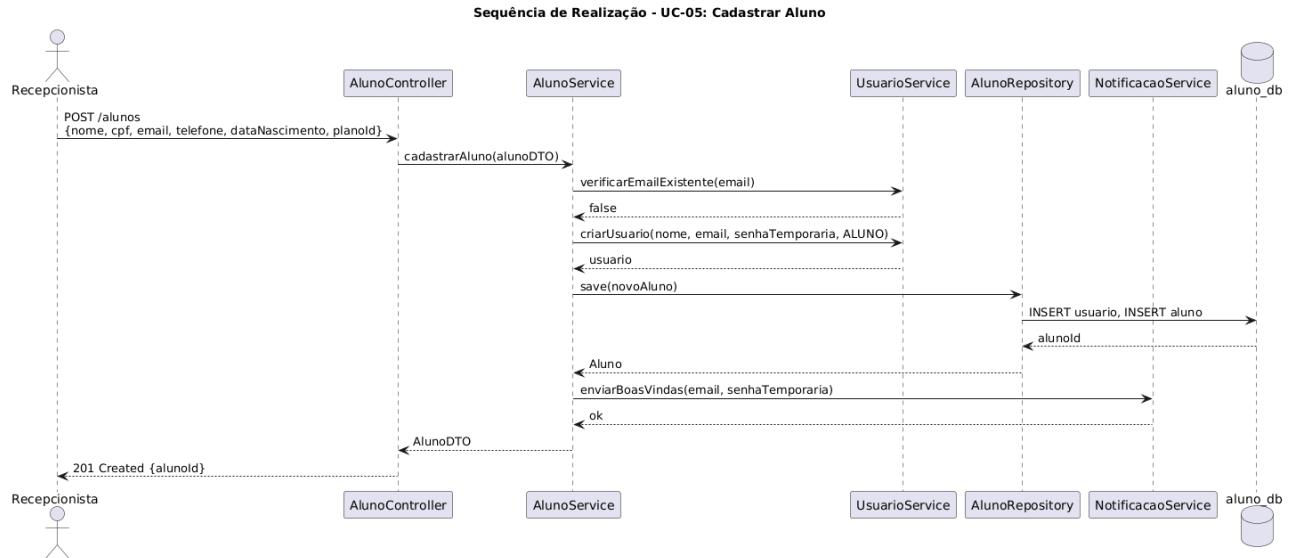
Sequência de Realização — UC-01: Realizar Login



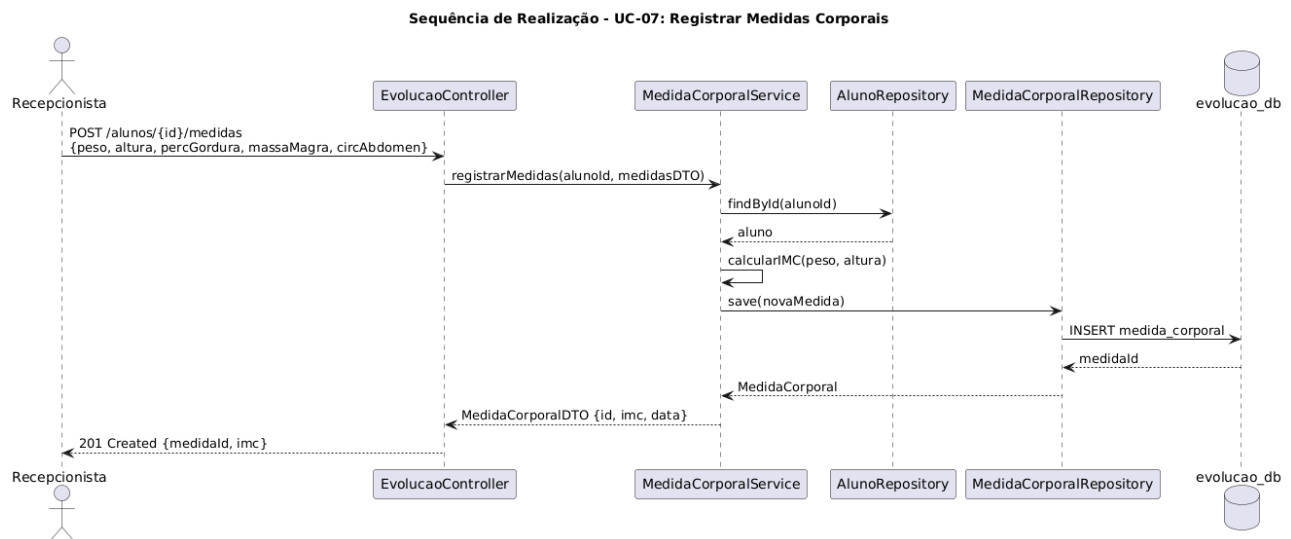
Sequência de Realização — UC-03: Registrar Execução de Treino



Sequência de Realização — UC-05: Cadastrar Aluno



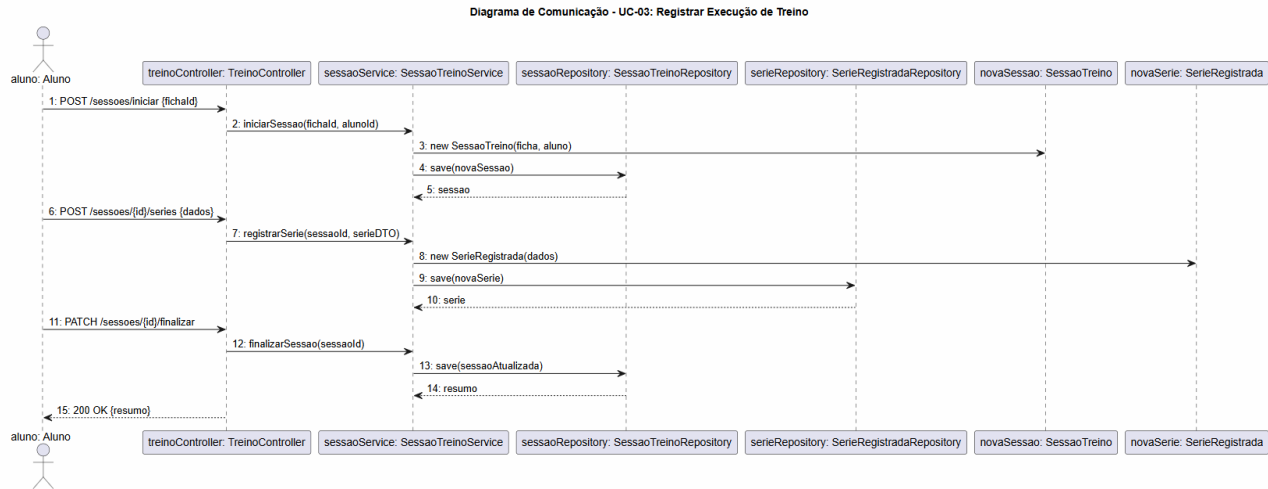
Sequência de Realização — UC-07: Registrar Medidas Corporais



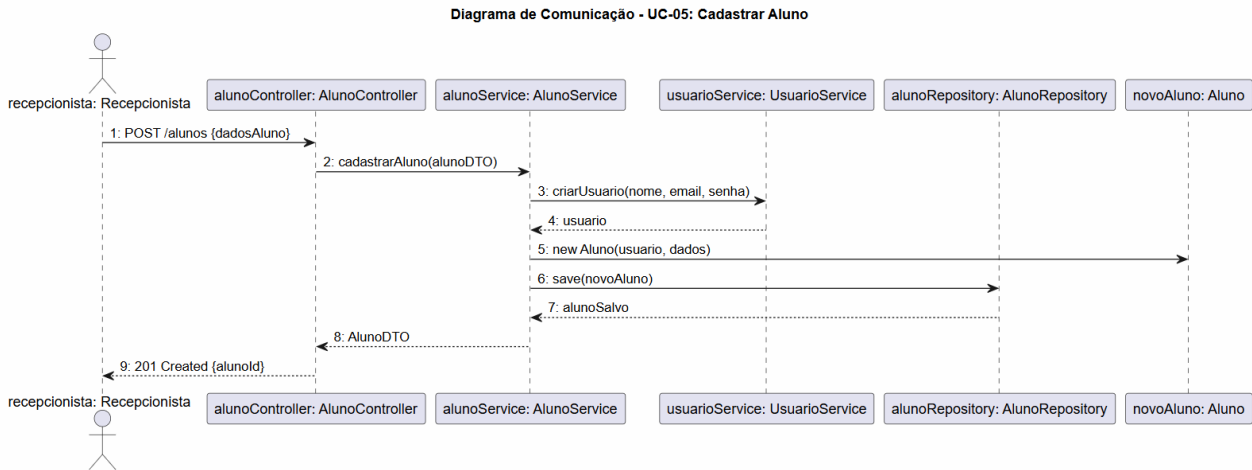
3.5 Diagramas de Comunicação (Os diagramas de comunicação foram feitos com template dos diagramas de sequência mediante autorização do professor)

Os diagramas de comunicação mostram as interações entre objetos para realização dos casos de uso, com a sequência numerada das mensagens trocadas.

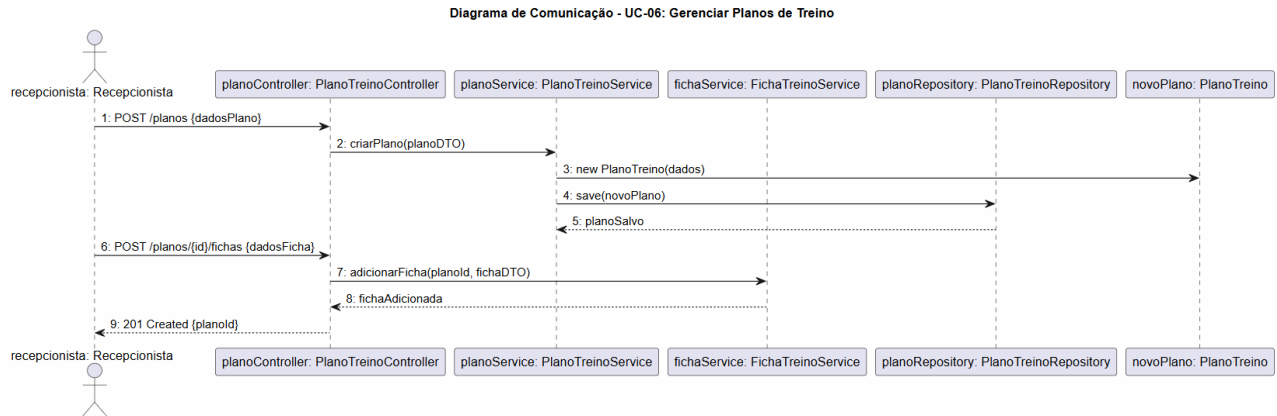
Comunicação — UC-03: Registrar Execução de Treino



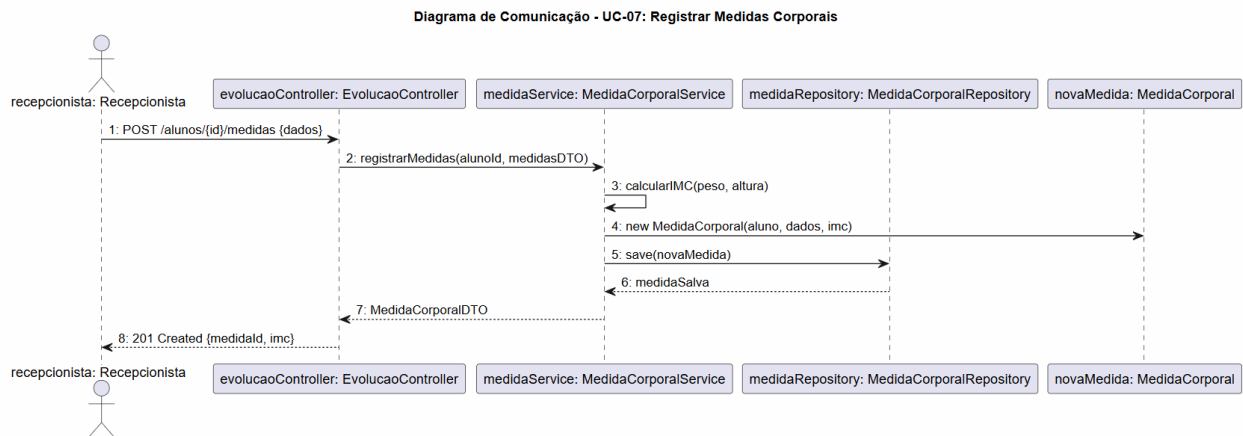
Comunicação — UC-05: Cadastrar Aluno



Comunicação — UC-06: Gerenciar Planos de Treino



Comunicação — UC-07: Registrar Medidas Corporais

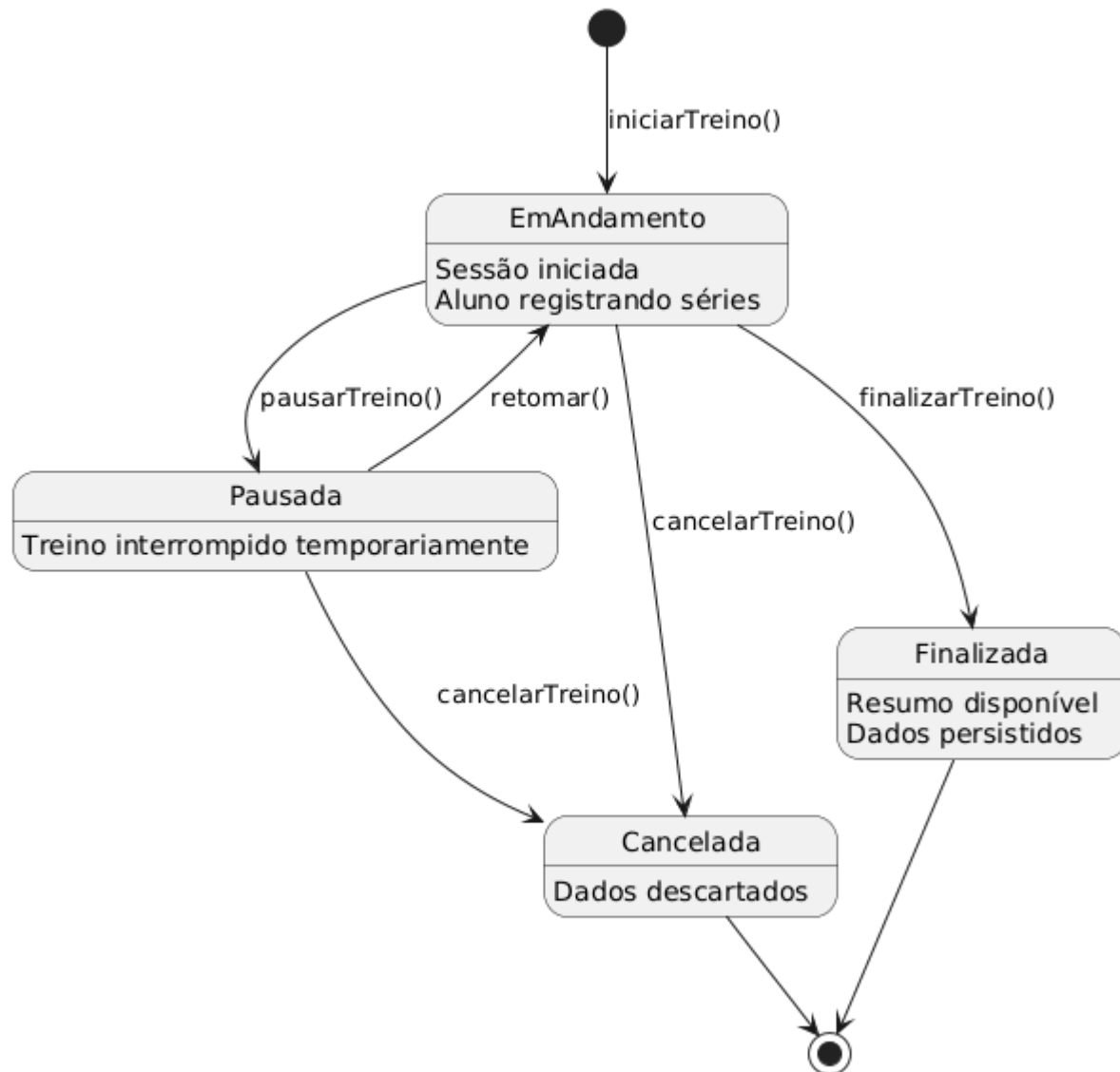


3.6 Diagramas de Estados

Os diagramas de estados modelam os ciclos de vida dos principais objetos do sistema.

Estados da Sessão de Treino

Diagrama de Estados - Sessão de Treino



Estados do Aluno

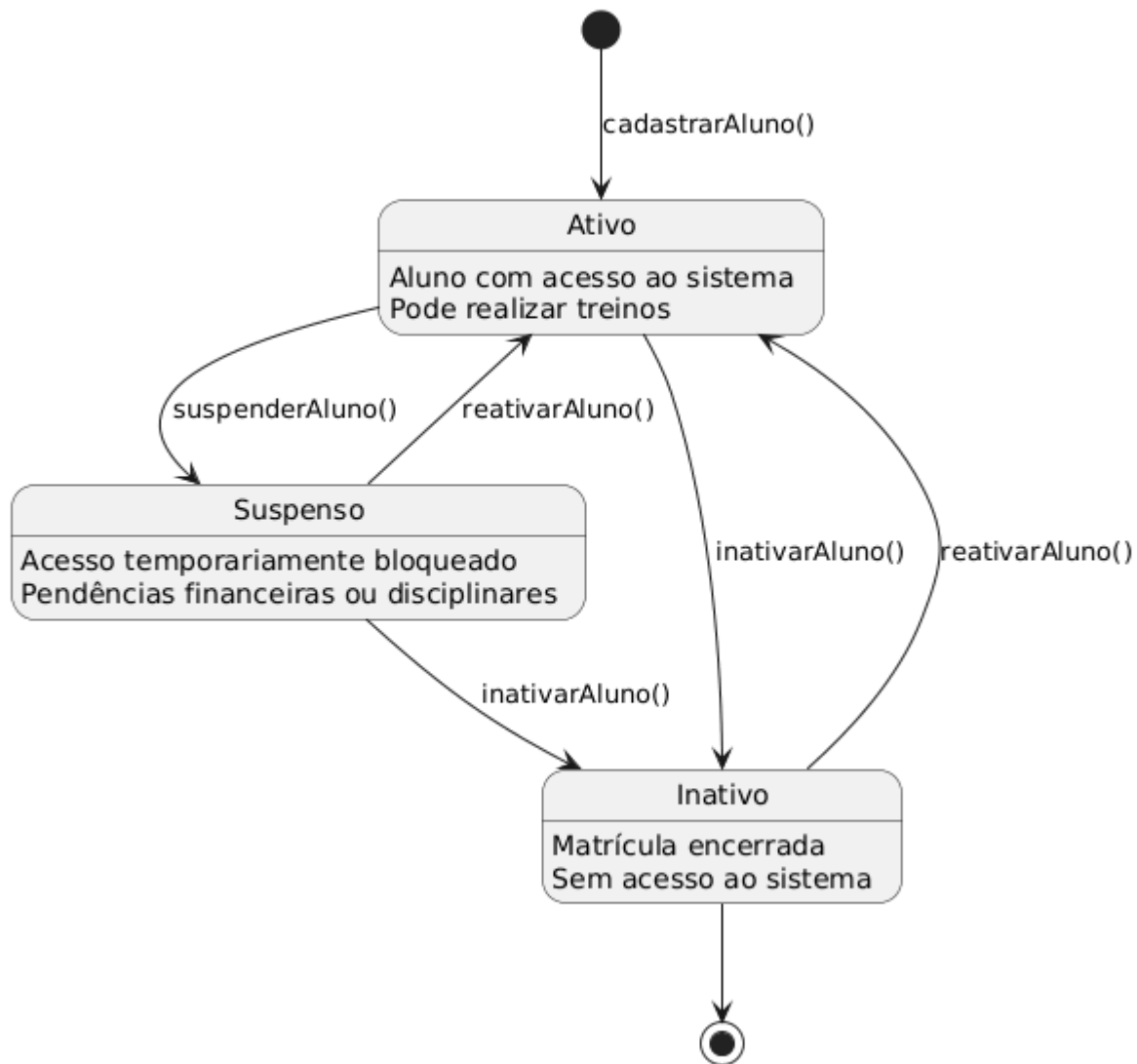
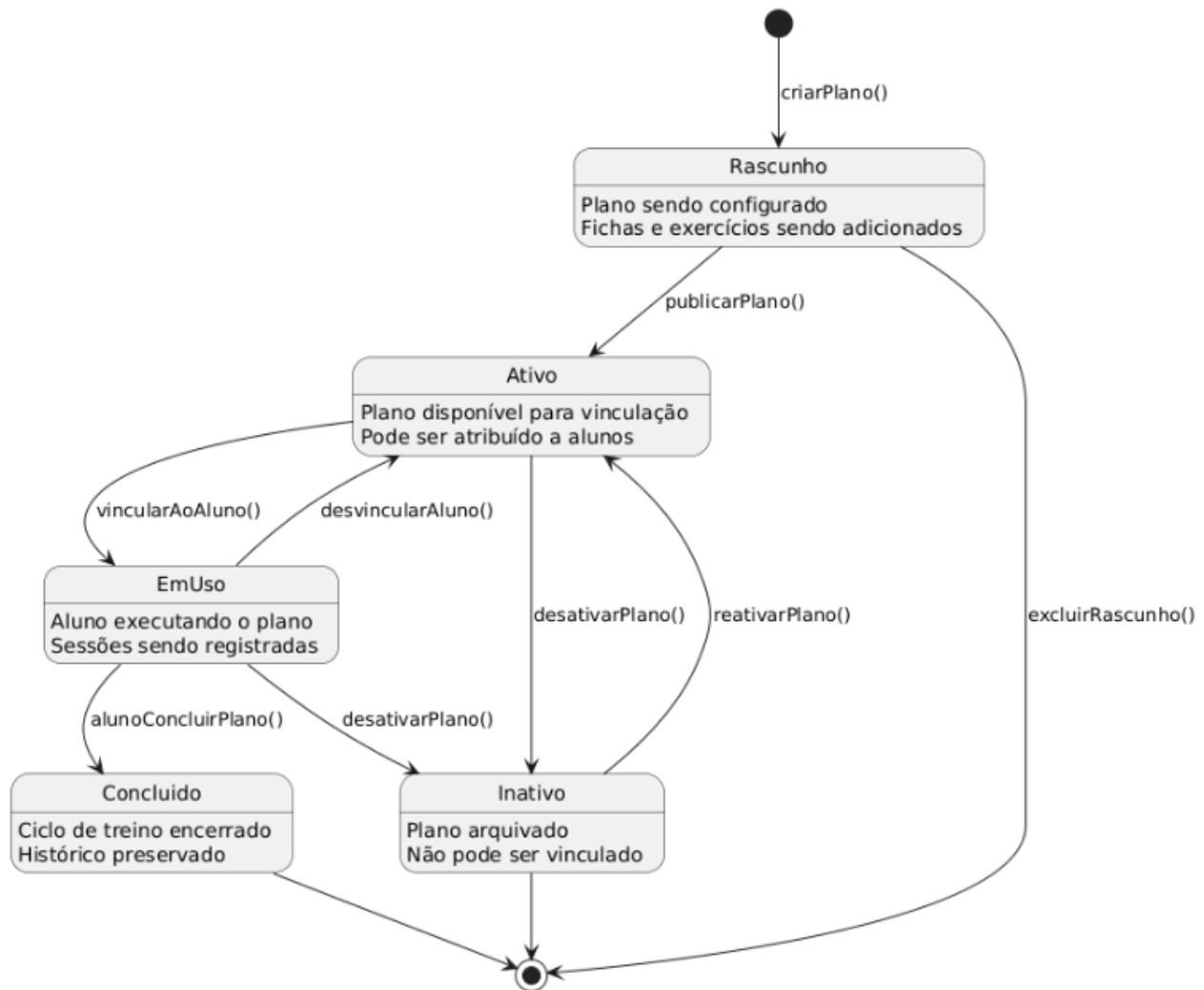
Diagrama de Estados - Aluno**Estados do Plano de Treino**

Diagrama de Estados - Plano de Treino



4. Modelos de Dados

O banco de dados do GymFlow utiliza PostgreSQL 16. Cada microserviço possui seu próprio schema isolado seguindo o padrão Database per Service. O mapeamento objeto-relacional é realizado via Hibernate/JPA com as seguintes estratégias:

- Herança: não utilizada; cada entidade mapeada para sua própria tabela (Table per Class).
- Relacionamentos: @OneToMany e @ManyToOne com fetch LAZY por padrão para evitar N+1.
- IDs: gerados por sequência no banco (BIGSERIAL / GenerationType.IDENTITY).
- Auditoria: campos created_at e updated_at gerenciados via @EntityListeners e @CreateDate/@LastModifiedDate.
- Índices: criados automaticamente nas chaves estrangeiras e nos campos email e cpf da tabela usuario.

Diagrama Entidade-Relacionamento (DER) (PRÓXIMA PÁGINA)

Modelo de Dados - GymFlow

